colegio
CALASANZ
SALAMANCA

Paseo de Canalejas, 139-159
37001 Salamanca
Tel.: 923 26 79 61 | Fax: 923 27 04 54
www.calasanzsalamanca.es
CENTRO PRIVADO CONCERTADO

[Adopta Un Compañero]

CFGS DAW

CURSO 2025/2026

[Diego Larrazábal Mendoza e Ivan Sánchez Dominguez]

COLEGIO CALASANZ SALAMANCA

Índice

1. Introducción3
2. Descripción de la aplicación.4
3. Plan de empresa11
4. Tecnologías escogidas y justificación17
5. Diseño de la aplicación19
   5.1. Diagramas y definición de casos de uso19
   5.2. Diagramas de clase23
   5.3. Modelo entidad relación26
6. Arquitectura de la aplicación28
   6.1. Estructura del proyecto29
   6.2. Librerías externas utilizadas31
7. Manual de despliegue33

1. Introducción

Temática: **Centro de adopción de animales**

Nombre: **AdoptaUnCompañero**

Introducción:

AdoptaUnCompañero será una plataforma web dedicada a facilitar la adopción responsable de animales que buscan un nuevo hogar.

Nuestro objetivo es conectar a personas dispuestas a brindar amor y cuidados con aquellos animales que lo necesitan, ofreciendo información clara, actualizada y accesible sobre cada uno de ellos. A través de esta web, los usuarios podrán conocer a los animales disponibles, sus características y necesidades, así como iniciar el proceso de adopción de forma sencilla y segura.

2. Descripción de la aplicación.

2.1 Objetivo de la aplicación

Está destinada a la gestión de adopciones de animales, el objetivo principal es poner en contacto a posibles adoptantes con animales que buscan un nuevo hogar, facilitando el proceso de una adopción controlado, responsable y transparente.

La aplicación permitirá:

- Mostrar de forma clara los animales disponibles
- Gestionar solicitudes de adopción.
- Realizar un seguimiento completo del proceso
- Facilitar la labor administrativa del centro de adopción

2.2 Tipos de usuarios y roles

- Cliente (usuario no registrado)

Es el usuario que accede a la aplicación sin haber iniciado sesión.

- 2.2.1 Funcionalidades:

- 2.2.1.1 Acceder a la página de inicio
- 2.2.1.2 Consultar el catálogo de animales disponibles
- 2.2.1.3 Visualizar la ficha detallada de cada animal
- 2.2.1.4 Consultar la información sobre el proceso de adopción
- 2.2.1.5 Acceder al formulario de registrarse o iniciar sesión

- 2.2.2 Restricciones:
- 2.2.2.1 No puede iniciar procesos de adopción

- 2.2.2.2 No puede guardar animales ni acceder a un área personal.

- Cliente registrado (adoptante)

Usuario que ha creado una cuenta en la aplicación

- Funcionalidades:
- 2.2.3.1 Iniciar y cerrar sesión.
- 2.2.3.2 Acceder a un área personal.
- 2.2.3.3 Guardar animales como favoritos.
- 2.2.3.4 Iniciar solicitudes de adopción.
- 2.2.3.5 Consultar el estado de sus solicitudes.
- 2.2.3.6 Gestionar sus datos personales.
- 2.2.3.7 Cancelar una solicitud mientras esté en estado inicial

- Administrador (personal del centro)

Usuario encargado de la gestión interna del centro de adopción.

- Funcionalidades:
- 2.2.4.1 Gestionar animales (crear, modificar, eliminar).
- 2.2.4.2 Gestionar solicitudes de adopción.
- 2.2.4.3 Cambiar el estado de una solicitud.
- 2.2.4.4 Programar citas con los adoptantes.
- 2.2.4.5 Finalizar procesos de adopción.
- 2.2.4.6 Gestionar usuarios registrados.
- 2.2.4.7 % de animales adoptados o qué animales se adoptan más gestionados internamente

El administrador podrá consultar estadísticas generadas automáticamente por el sistema,

como el porcentaje de animales adoptados, los tipos de animales más adoptados, el tiempo

medio de duración de los procesos de adopción y el número de solicitudes aprobadas o

rechazadas. Estas estadísticas permitirán al centro analizar su actividad y mejorar el

proceso de adopción.

2.2.5 Funcionalidades con lógica de negocio

Entre estas funcionalidades se incluyen:

- Envío automático de notificaciones por correo electrónico al cliente cuando se produzcan cambios en el estado de su solicitud de adopción.
- Registro del historial de cambios de estado de cada solicitud, permitiendo un seguimiento del proceso.
- Generación de estadísticas internas basadas en los datos almacenados en el sistema.

2.3 Módulos y pantallas de la aplicación.

Página de inicio: Pantalla principal que presenta la aplicación y su finalidad.

- Funcionalidades:

- Mostrar información general del proyecto.
- Mostrar un resumen de animales disponibles.
- Acceso directo al catálogo de animales.

- Acceso a registro e inicio de sesión.
- Acceso a información sobre el proceso de adopción.

2.3.1 Catálogo de animales: Pantalla donde se muestran todos los animales disponibles para adopción.

- Funcionalidades:

- 2.3.1.0: Listado de animales con información básica:
- 2.3.1.1: Imagen
- 2.3.1.2: Nombre
- 2.3.1.3: Especie
- 2.3.1.4: Edad aproximada

- Sistema de filtros:
- 2.3.1.5: Especie
- 2.3.1.6: Edad
- 2.3.1.7: Tamaño
- 2.3.1.8: Estado del animal
- 2.3.1.9: Búsqueda por nombre
- 2.3.1.10: Acceso a la ficha detallada del animal

2.3.2 Ficha del animal: Pantalla con la información completa de un animal concreto.

- Funcionalidades:

- 2.3.2.1: Visualización de imágenes del animal.
- 2.3.2.2: Información detallada:
- Características físicas
- Personalidad
- Necesidades especiales
- Estado sanitario
- 2.3.2.3: Indicación del estado del animal:
- Disponible
- Reservado
- Adoptado
- 2.3.2.4: Opciones disponibles:
- Guardar como favorito (cliente registrado).
- Iniciar proceso de adopción (si está disponible).

2.3.3 Área personal del cliente: Espacio privado para los clientes registrados.

- 2.3.3.1 Funcionalidades:
  - Visualizar información personal.
  - Editar datos de contacto.
  - Consultar animales marcados como favoritos.
  - Acceder al historial de solicitudes de adopción.
  - Consultar el estado actual de cada solicitud.

2.3.4 Proceso de adopción: Conjunto de pantallas que gestionan la solicitud de adopción.

- Funcionalidades:

1. Inicio del proceso desde la ficha del animal.

2. Relleno de un formulario de adopción con información personal y del entorno del adoptante.

3. Envío de la solicitud al centro.

4. Asignación de un estado a la solicitud:

- En revisión
- Aprobada
- Rechazada
- En proceso
- Finalizada

5. Seguimiento del proceso desde el área personal del cliente.

- Cada cambio de estado de una solicitud generará una acción automática por parte del

sistema, como el envío de una notificación al cliente y el registro del cambio en el historial de la adopción, garantizando el seguimiento completo del proceso.

2.3.5 Área de administración: Espacio privado destinada al personal del centro.

- Funcionalidades:

- Panel de control con resumen de la actividad.
- Gestión completa de animales.

1. Gestión de solicitudes:

1.1 Visualización de datos del solicitante.

1.2 Aprobación o rechazo de solicitudes.

1.3 Cambio de estado de las solicitudes.

- Gestión de citas de adopción.
- Control de usuarios registrados.

2.4 Flujo general y resumido del funcionamiento de la aplicación

- 2.4.1: El cliente accede a la aplicación.
- 2.4.2: Consulta el catálogo de animales.
- 2.4.3: Visualiza la ficha de un animal.
- 2.4.4: Se registra o inicia sesión.
- 2.4.5: Inicia una solicitud de adopción.
- 2.4.6 El administrador revisa la solicitud y modifica su estado.
- 2.4.7 El sistema aplica las reglas de negocio correspondientes y envía una notificación automática al cliente.
- 2.4.8 El proceso de adopción continúa hasta su finalización, quedando registrado todo el historial del proceso.
- 2.4.9 El cliente puede consultar en todo momento el estado y el historial de su solicitud desde su área personal.

3. Plan de empresa

AdoptaUnCompañero nace con el propósito de transformar la forma en que los centros de protección animal gestionan sus adopciones. Más allá de un proyecto formativo, la aplicación tiene un recorrido real como producto: hoy en día la mayoría de protectoras siguen gestionando sus animales y solicitudes con hojas de cálculo o herramientas genéricas, sin un sistema pensado específicamente para ellas. Ahí es donde entra nuestro proyecto.

3.3.1. Viabilidad técnica

La aplicación está construida sobre tecnologías maduras y ampliamente respaldadas por la industria: **Spring Boot** en el backend, **Vue.js** en el frontend y **MySQL** como base de datos, todas ellas de código abierto. Esto significa cero costes de licencia y la garantía de que el proyecto contará con soporte y mejoras durante muchos años.

La arquitectura cliente-servidor desacoplada permite que la aplicación crezca sin sobresaltos: añadir una app móvil, integrar pasarelas de pago para donaciones o conectar con clínicas veterinarias es totalmente posible sin tocar el núcleo del sistema. El despliegue mediante **Docker** facilita instalarla en cualquier servidor en cuestión de minutos.

3.3.2. Viabilidad económica

El modelo de negocio propuesto es **SaaS (Software as a Service)**: el centro paga una cuota mensual y a cambio recibe la aplicación funcionando, el alojamiento, las actualizaciones y el soporte técnico. Sin instalaciones complicadas ni inversión inicial por su parte.

Los costes de operación son muy bajos: un servidor cloud básico (entre 10 y 20€ al mes) es suficiente para dar servicio a varios centros simultáneamente, lo que permite ofrecer cuotas asequibles (entre 30 y 50€ mensuales) que cualquier protectora puede asumir dentro de su presupuesto. Esta estructura genera un margen amplio desde los primeros clientes.

A esto se suman fuentes de ingresos complementarias como **patrocinios** de clínicas veterinarias y tiendas de alimentación animal, que ven en la plataforma una vía directa para llegar a un público objetivo claramente identificado: personas que acaban de adoptar.

3.3.3. Viabilidad financiera

El mercado potencial en España supera las **700 protectoras y centros de adopción** registrados, sin contar ayuntamientos y asociaciones más pequeñas. Captando apenas un 5% de ese mercado en los primeros años, el proyecto sería plenamente rentable.

Además, el carácter social de la aplicación abre la puerta a **subvenciones específicas** que no están disponibles para proyectos comerciales convencionales: ayudas de la Fundación Affinity, programas autonómicos de bienestar animal y fondos europeos orientados a la protección animal. Estas líneas de financiación pueden cubrir gran parte de la inversión inicial sin comprometer la propiedad del proyecto.

La planificación del proyecto **AdoptaUnCompañero** se ha diseñado teniendo en cuenta la experiencia adquirida durante el Grado Superior de Desarrollo de Aplicaciones Web, así como la complejidad y alcance de la aplicación a desarrollar.

Se plantea una **metodología incremental**, donde el proyecto se divide en fases claramente diferenciadas. Cada fase incluye tanto el desarrollo técnico de la aplicación como la elaboración de la memoria asociada, permitiendo un control continuo del progreso y facilitando la detección temprana de posibles mejoras.

La temporalización es **orientativa**, pero establece hitos claros que guían el desarrollo completo del proyecto.

3.1 Fase 1 – Análisis y definición del proyecto

Duración estimada: 3 semanas

Fechas aproximadas: 15 de enero – 4 de febrero

Objetivos:

- Definir de forma definitiva los requisitos funcionales y no funcionales.
- Establecer el alcance exacto del proyecto.
- Sentar las bases para el diseño y desarrollo posterior.

Tareas:

- Definición de la temática, nombre y objetivos de la aplicación.
- Identificación de usuarios, roles y restricciones.
- Análisis detallado de funcionalidades por módulo.
- Definición del flujo general de la aplicación.
- Redacción de los apartados 1 y 2 de la memoria.
- Validación de los requisitos con el tutor del proyecto.

Hito:

- Requisitos cerrados y aprobados.

3.2 Fase 2 – Diseño de la aplicación

Duración estimada: 3 semanas.

Fechas aproximadas: 5 de febrero – 25 de febrero.

Objetivos:

- Diseñar la estructura técnica y visual de la aplicación.
- Definir la arquitectura del proyecto.

Tareas:

- Diseño de la estructura de carpetas del proyecto.
- Diseño de la base de datos (entidades, relaciones y estados).
- Elaboración de prototipos de las principales pantallas:
  - Página de inicio.
  - Catálogo de animales.
  - Ficha del animal.
  - Área personal del cliente.
  - Área de administración.
- Definición de la navegación y flujo entre pantallas.
- Redacción del apartado de diseño en la memoria.

Hito:

- Diseño completo validado antes de comenzar la implementación.

3.3 Fase 3 – Desarrollo del frontend

Duración estimada: 4 semanas.

Fechas aproximadas: 26 de febrero – 24 de marzo.

Objetivos:

- Implementar la interfaz de usuario de forma funcional y responsive.
- Garantizar una experiencia de usuario clara e intuitiva.

Tareas:

- Maquetación de la página de inicio.
- Implementación del catálogo de animales con filtros y búsqueda.
- Desarrollo de la ficha detallada del animal.
- Formularios de registro e inicio de sesión.
- Área personal del cliente (datos, favoritos, solicitudes).
- Integración inicial con datos simulados o de prueba.

Hito:

- Frontend completamente funcional y navegable.

3.4 Fase 4 – Desarrollo del backend

Duración estimada: 5 semanas.

Fechas aproximadas: 25 de marzo – 28 de abril.

Objetivos:

- Implementar la lógica de negocio y la gestión de datos.
- Asegurar el correcto funcionamiento de los procesos de adopción.

Tareas:

- Implementación del sistema de autenticación y roles.
- Gestión de usuarios (clientes y administradores).
- CRUD completo de animales.
- Gestión de solicitudes de adopción y cambios de estado.
- Implementación del sistema de favoritos.
- Programación del área de administración.
- Gestión de citas y finalización de adopciones.
- Integración completa frontend y backend.

Hito:

- Aplicación completamente operativa a nivel funcional.

3.5 Fase 5 – Pruebas y validación

Duración estimada: 2 semanas.

Fechas aproximadas: 29 de abril – 12 de mayo.

Objetivos:

- Garantizar la estabilidad, seguridad y correcto funcionamiento de la aplicación.

Tareas:

- Pruebas funcionales por módulo.
- Pruebas de flujos completos de adopción.
- Verificación de permisos según rol de usuario.
- Corrección de errores detectados.
- Optimización de rendimiento y usabilidad.
- Ajustes finales según feedback del tutor.

Hito:

- Aplicación estable y lista para su despliegue.

3.6 Fase 6 – Documentación final y despliegue

Duración estimada: 2 semanas.

Fechas aproximadas: 13 de mayo – 27 de mayo.

Objetivos:

- Finalizar la memoria y preparar la entrega del TFG.

Tareas:

- Redacción final de la memoria del proyecto.

- Inclusión de capturas y explicaciones técnicas.
- Documentación de decisiones de diseño y desarrollo.
- Preparación del entorno de despliegue.
- Subida del proyecto a repositorio (Git).
- Preparación de la defensa del proyecto.

Trabajo final terminado del proyecto: 27 de mayo (fecha provisional).

Hito:

- Proyecto y memoria finalizados y entregados.

3.7 Resumen de la planificación

Fase	**Duración**
Análisis	3 semanas
Diseño	3 semanas
Frontend	4 semanas
Backend	5 semanas
Pruebas	2 semanas
Documentación y despliegue	2 semanas
Total estimado	**19 semanas**

4. Tecnologías escogidas y justificación

Para el desarrollo del proyecto AdoptaUnCompañero se ha optado por una arquitectura cliente-servidor desacoplada, eligiendo en cada capa las tecnologías más adecuadas en cuanto a madurez, soporte de la comunidad y adecuación a los conocimientos adquiridos durante el ciclo formativo.

4.1.1. Backend

Java 17 y Spring Boot 3: Java 17 es una versión LTS, estable y con soporte a largo plazo. Spring Boot es el framework más utilizado actualmente en el desarrollo de aplicaciones empresariales en Java. Permite crear aplicaciones autocontenidas con servidor Tomcat embebido y reduce drásticamente la configuración gracias a su sistema de starters y autoconfiguración. Es, además, la tecnología estudiada en el módulo de Desarrollo Web en Entorno Servidor.

Spring Data JPA e Hibernate: Permiten trabajar con la base de datos a través de objetos Java en lugar de escribir SQL manualmente, generando automáticamente las consultas a partir del nombre de los métodos del repositorio.

Spring Security y JWT: Spring Security es el estándar de facto para gestionar la autenticación y autorización en aplicaciones Spring. Combinado con JWT (JSON Web Tokens), permite mantener un sistema stateless adecuado para una API REST, donde el servidor no necesita guardar información de sesión.

Maven: Herramienta para la gestión de dependencias y la construcción del proyecto, que permite definir todas las librerías necesarias en un único fichero pom.xml.

Jakarta Mail: Para el envío automático de notificaciones por correo electrónico cuando cambia el estado de una solicitud de adopción.

4.1.2. Base de datos

MySQL 8: SGBD relacional de código abierto, ampliamente utilizado y con un excelente rendimiento. La estructura de los datos del proyecto (usuarios, animales, solicitudes, citas) encaja perfectamente con el modelo relacional, donde las relaciones entre entidades se gestionan mediante claves foráneas.

4.1.3. Frontend

Vue.js 3 y Vite: Vue.js es un framework progresivo de JavaScript que permite construir interfaces de usuario reactivas mediante componentes reutilizables. Se ha elegido frente a React o Angular por su curva de aprendizaje suave y la sencillez de su sintaxis. Vite es la herramienta de build oficial recomendada para Vue, con tiempos de arranque casi instantáneos.

Vue Router y Axios: Vue Router gestiona la navegación entre las distintas vistas de la SPA sin recargar la página. Axios es el cliente HTTP utilizado para realizar las peticiones a la API REST del backend, con interceptores que añaden automáticamente el token JWT.

HTML5, CSS3 y JavaScript (ES6+): Tecnologías base de cualquier aplicación web, utilizadas para la estructura, los estilos y la lógica del lado del cliente.

4.1.4. Herramientas de desarrollo

Eclipse IDE (Spring Tool Suite): IDE utilizado para el desarrollo del backend, con soporte nativo para Spring Boot e integración con Maven.

Visual Studio Code: Editor ligero y extensible utilizado para el desarrollo del frontend, con extensiones específicas para Vue.

Git y GitHub: Sistema de control de versiones distribuido y plataforma de alojamiento, que permite que ambos integrantes del equipo trabajen de forma coordinada sobre el mismo código.

Bruno: Cliente de API REST de código abierto, alternativa moderna a Postman, utilizado para probar los endpoints durante el desarrollo.

MySQL Workbench: Herramienta para la administración visual de la base de datos durante el desarrollo.

Draw.io: Herramienta gratuita utilizada para la elaboración de los diagramas UML y el modelo entidad-relación del proyecto.

5. Diseño de la aplicación

5.1. Diagramas y definición de casos de uso

La aplicación cuenta con los siguientes actores:

- **Cliente**

Usuario que accede a la aplicación sin haber iniciado sesión. Puede consultar información general sobre el centro y los animales disponibles, pero no puede iniciar procesos de adopción.

- **Cliente registrado**

Usuario que dispone de una cuenta en el sistema y ha iniciado sesión. Puede realizar acciones relacionadas con el proceso de adopción y la gestión de su información personal.

- **Administrador**

Personal del centro de adopción encargado de la gestión interna de la aplicación. Tiene acceso a funciones de administración, control y supervisión del proceso de adopción.

- **Servicio de Email**

Actor externo que representa el sistema de envío de notificaciones por correo electrónico, utilizado para informar a los usuarios de los cambios en el estado de sus solicitudes.

2. Casos de uso del Cliente

El **Cliente** puede realizar las siguientes acciones:

- **Consultar catálogo**: permite visualizar el listado de animales disponibles para adopción.
- **Ver ficha de animal**: muestra la información detallada de un animal concreto, incluyendo características y estado.
- **Consultar proceso de adopción**: proporciona información general sobre los pasos necesarios para adoptar un animal.

- **Registro / Inicio de sesión**: permite al usuario crear una cuenta o acceder a la aplicación como cliente registrado.

3. Casos de uso del Cliente registrado

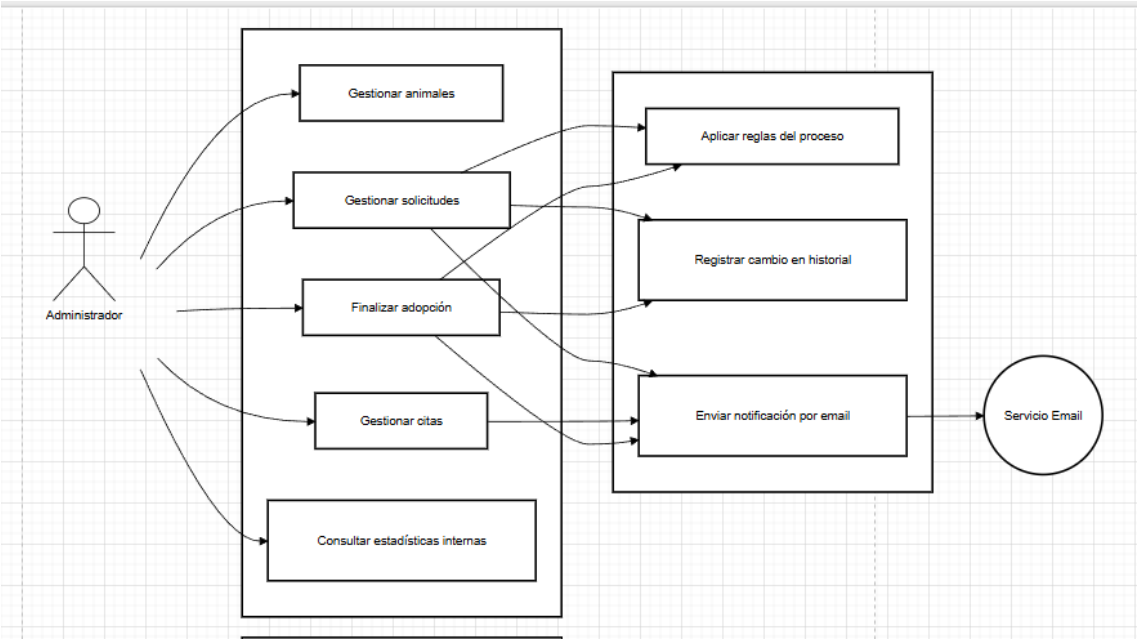
El **Cliente registrado** dispone de funcionalidades adicionales orientadas a la gestión del proceso de adopción:

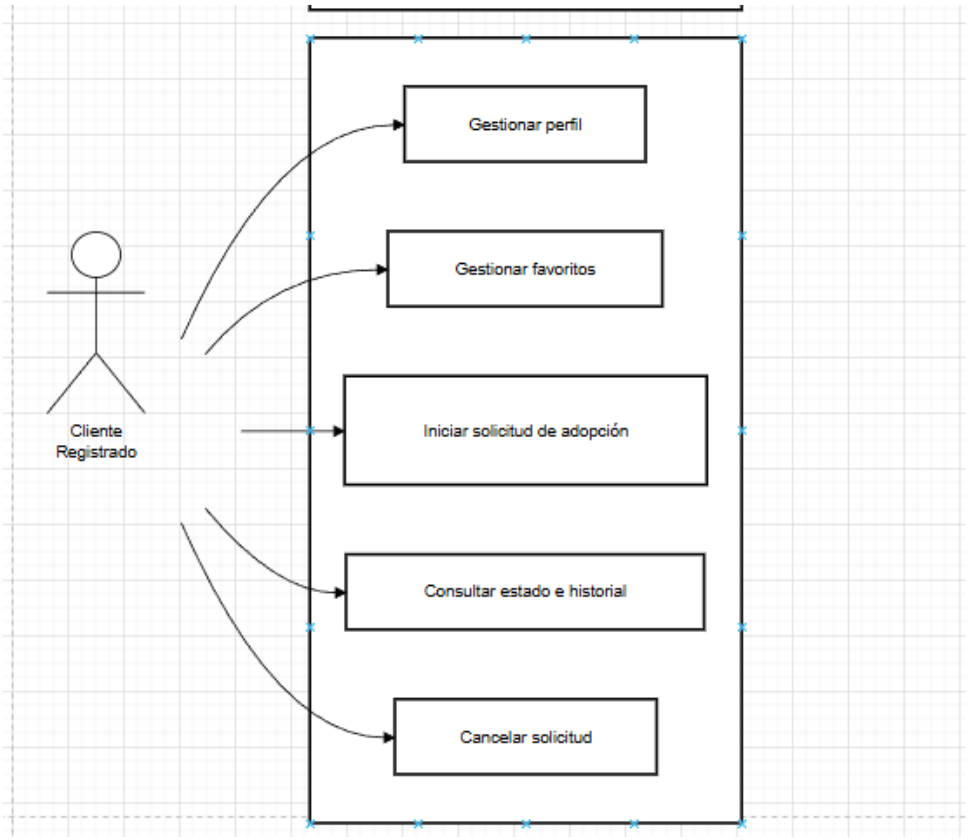
- **Gestionar perfil**: permite consultar y modificar los datos personales del usuario.
- **Gestionar favoritos**: posibilita guardar animales de interés y consultarlos posteriormente.
- **Iniciar solicitud de adopción**: permite comenzar el proceso de adopción de un animal disponible mediante el envío de una solicitud.
- **Consultar estado e historial**: permite visualizar el estado actual de una solicitud y el historial de cambios realizados durante el proceso.
- **Cancelar solicitud**: permite cancelar una solicitud de adopción mientras esta no haya sido finalizada.

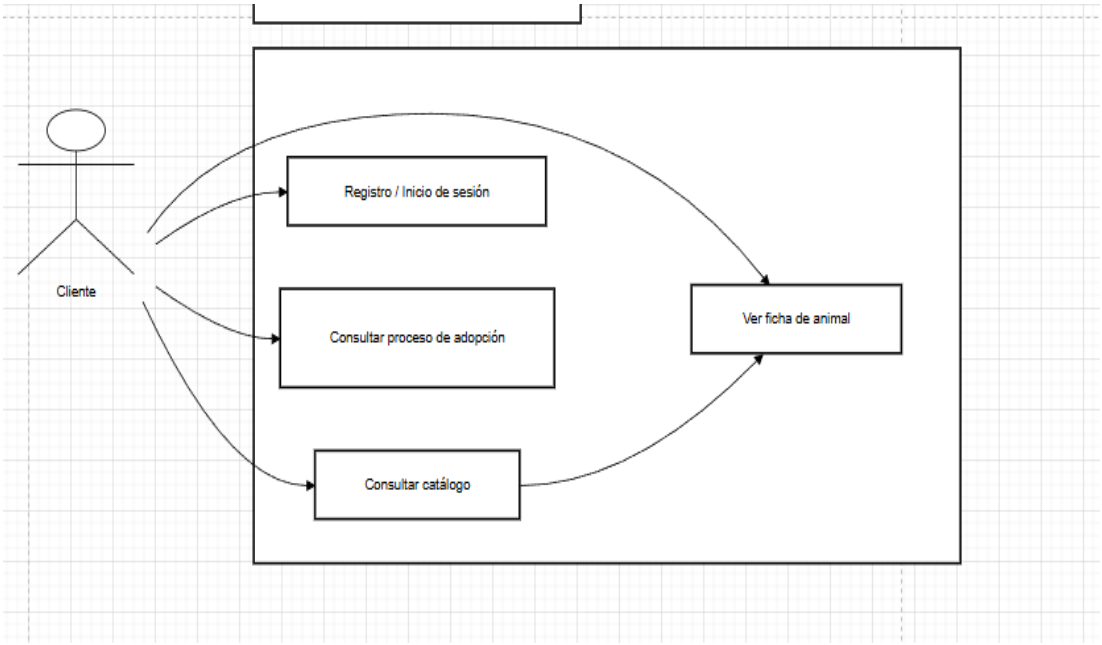
4. Casos de uso del Administrador

El **Administrador** es el actor responsable de la gestión interna del sistema y del correcto desarrollo del proceso de adopción:

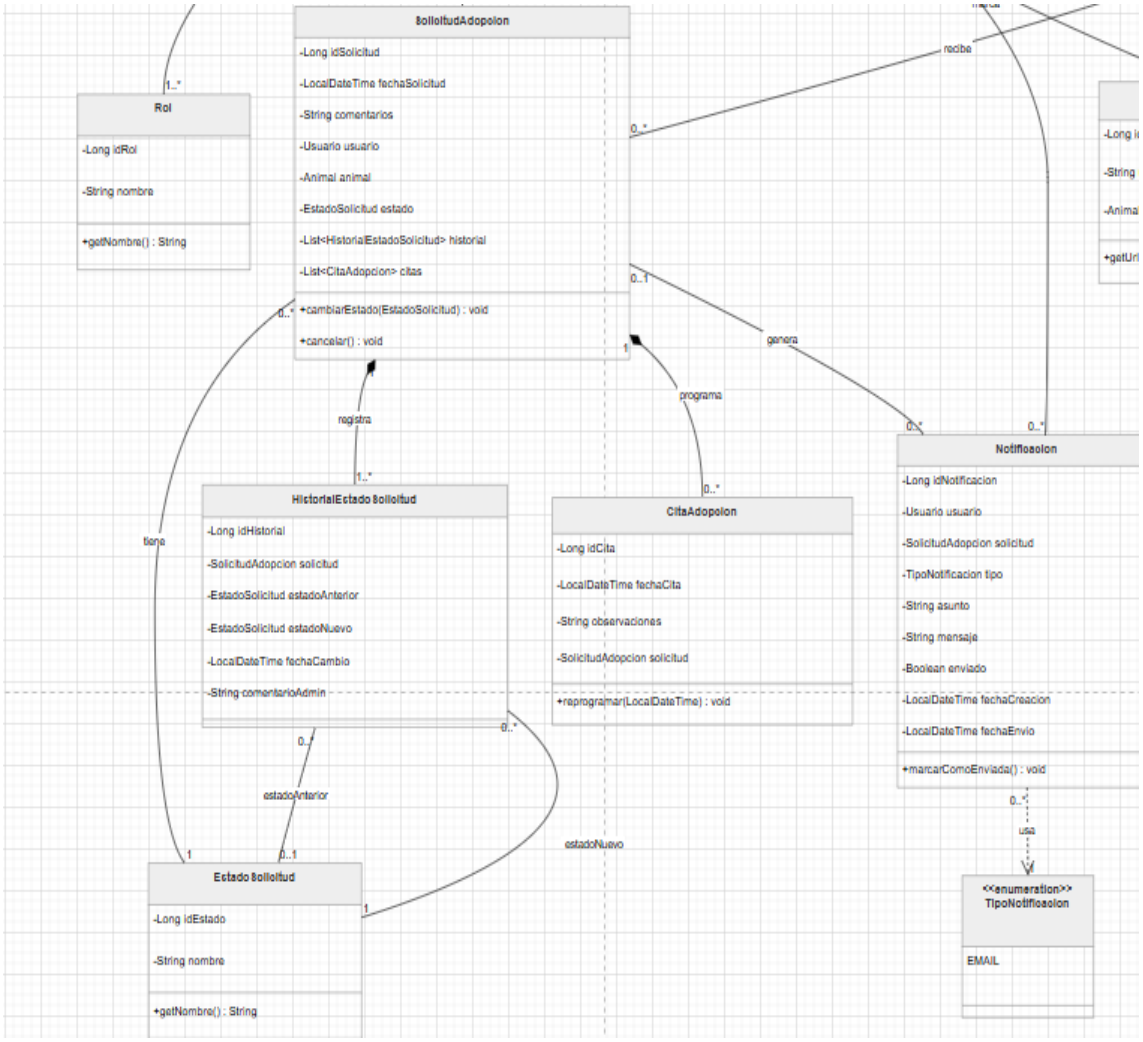
- **Gestionar animales**: permite crear, modificar, eliminar y cambiar el estado de los animales.
- **Gestionar solicitudes**: permite revisar las solicitudes recibidas y tomar decisiones sobre las mismas.
- **Gestionar citas**: permite programar, modificar o cancelar citas relacionadas con el proceso de adopción.
- **Finalizar adopción**: permite cerrar el proceso de adopción cuando este ha sido completado correctamente.
- **Consultar estadísticas internas**: permite acceder a estadísticas generadas por el sistema para analizar la actividad del centro.

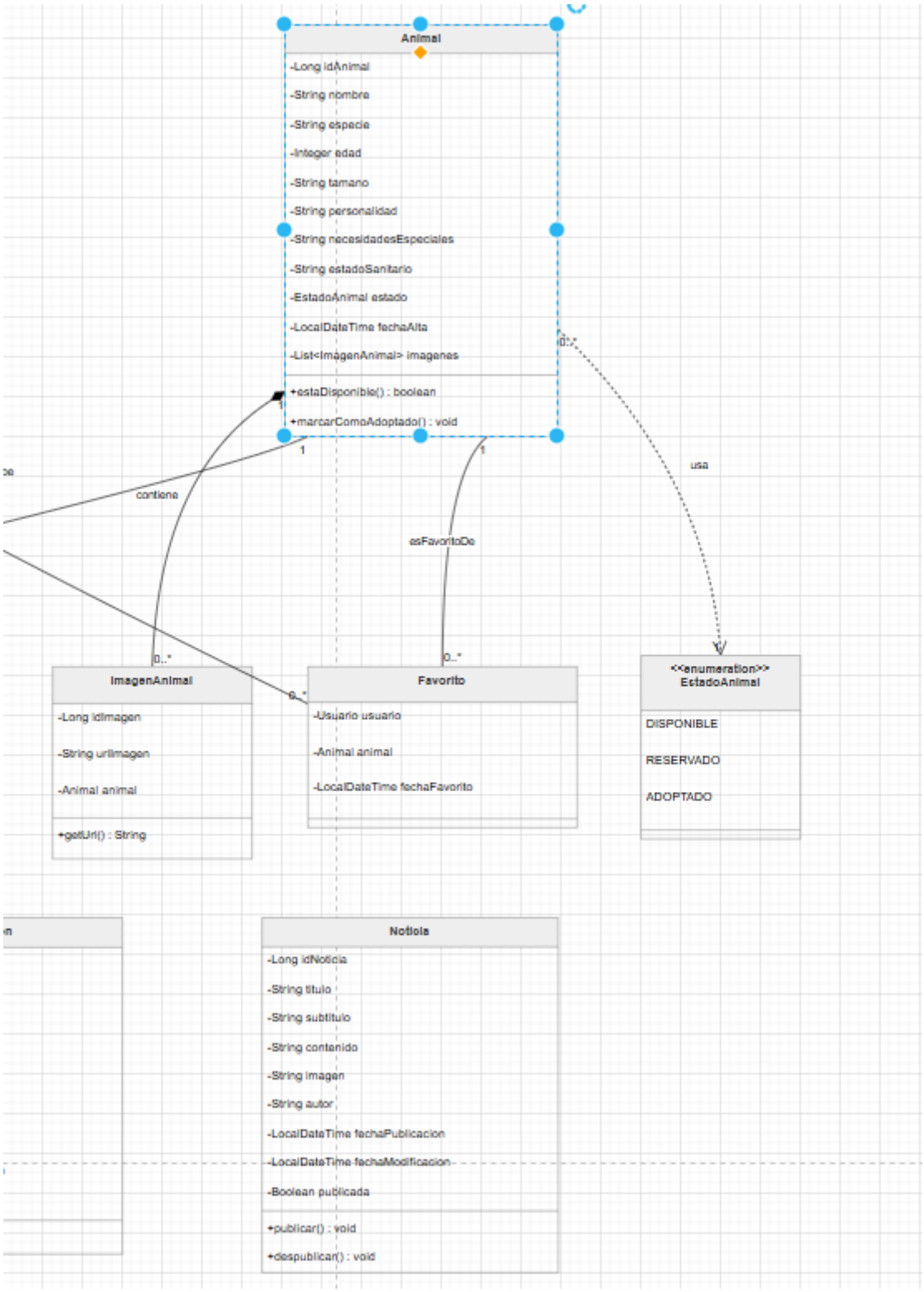

Administrador
Gestionar animales
Gestionar solicitudes
Finalizar adopción
Gestionar citas
Consultar estadísticas internas
Aplicar reglas del proceso
Registrar cambio en historial
Enviar notificación por email
Servicio Email

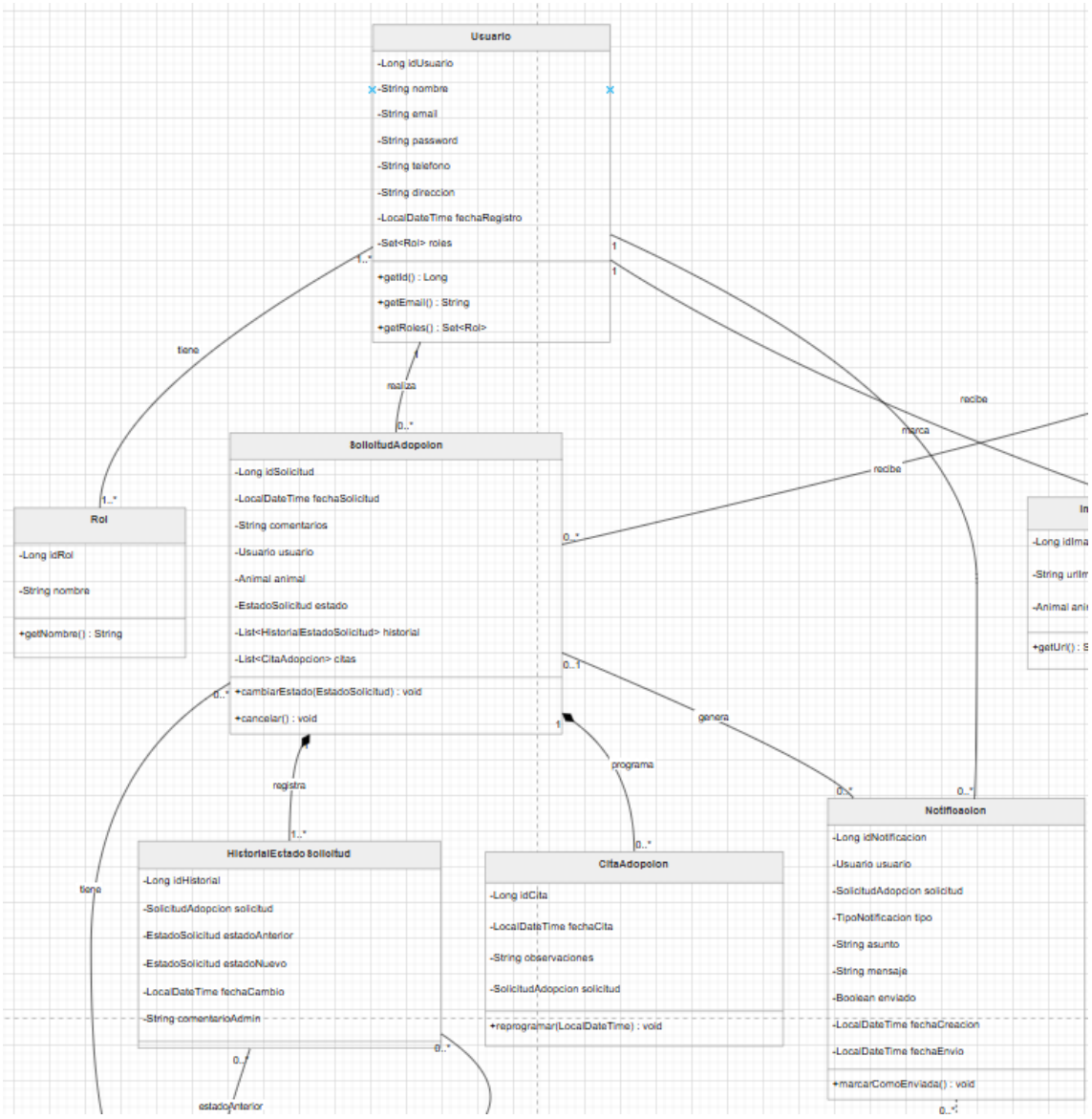

Cliente
Registrado
Gestionar perfil
Gestionar favoritos
Iniciar solicitud de adopción
Consultar estado e historial
Cancelar solicitud


Cliente
Registro / Inicio de sesión
Consultar proceso de adopción
Consultar catálogo
Ver ficha de animal

5.2. Diagramas de clase




Animal
-Long idAnimal
-String nombre
-String especie
-Integer edad
-String tamano
-String personalidad
-String necesidadesEspeciales
-String estadoSanitario
-EstadoAnimal estado
-LocalDateTime fechaAlta
-List<ImagenAnimal> imagenes
+estaDisponible() : boolean
+marcarComoAdoptado() : void
contiene
esFavoritoDe
usa
ImagenAnimal
-Long idImagen
-String urlImagen
-Animal animal
+getUrl() : String
Favorito
-Usuario usuario
-Animal animal
-LocalDateTime fechaFavorito
<<enumeration>>
EstadoAnimal
DISPONIBLE
RESERVADO
ADOPTADO
Noticia
-Long idNoticia
-String titulo
-String subtitulo
-String contenido
-String imagen
-String autor
-LocalDateTime fechaPublicacion
-LocalDateTime fechaModificacion
-Boolean publicada
+publicar() : void
+despublicar() : void



Habrá un archivo de draw.io para poder verlo completo.

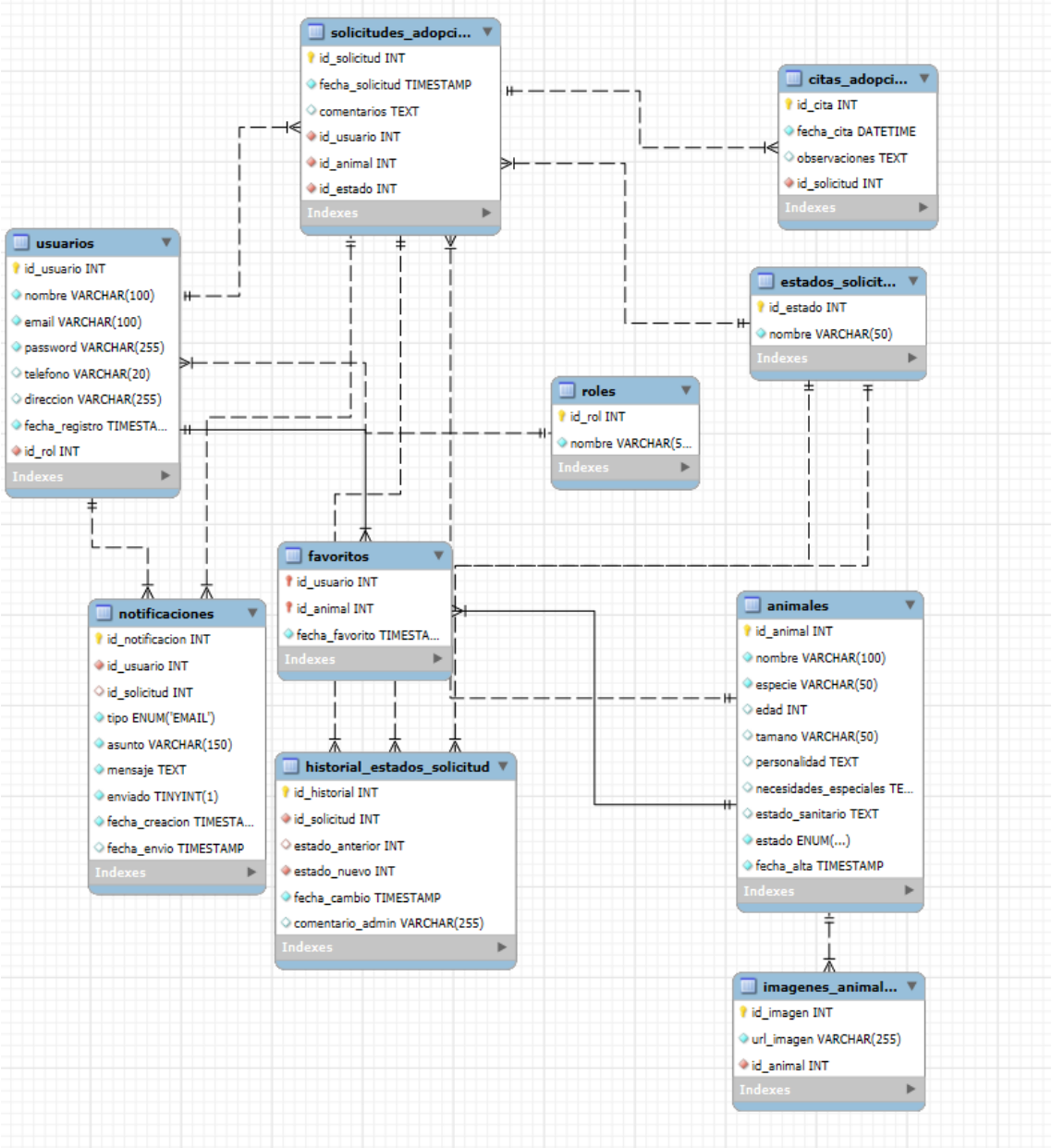
5.2.1. Cardinalidades del diagrama

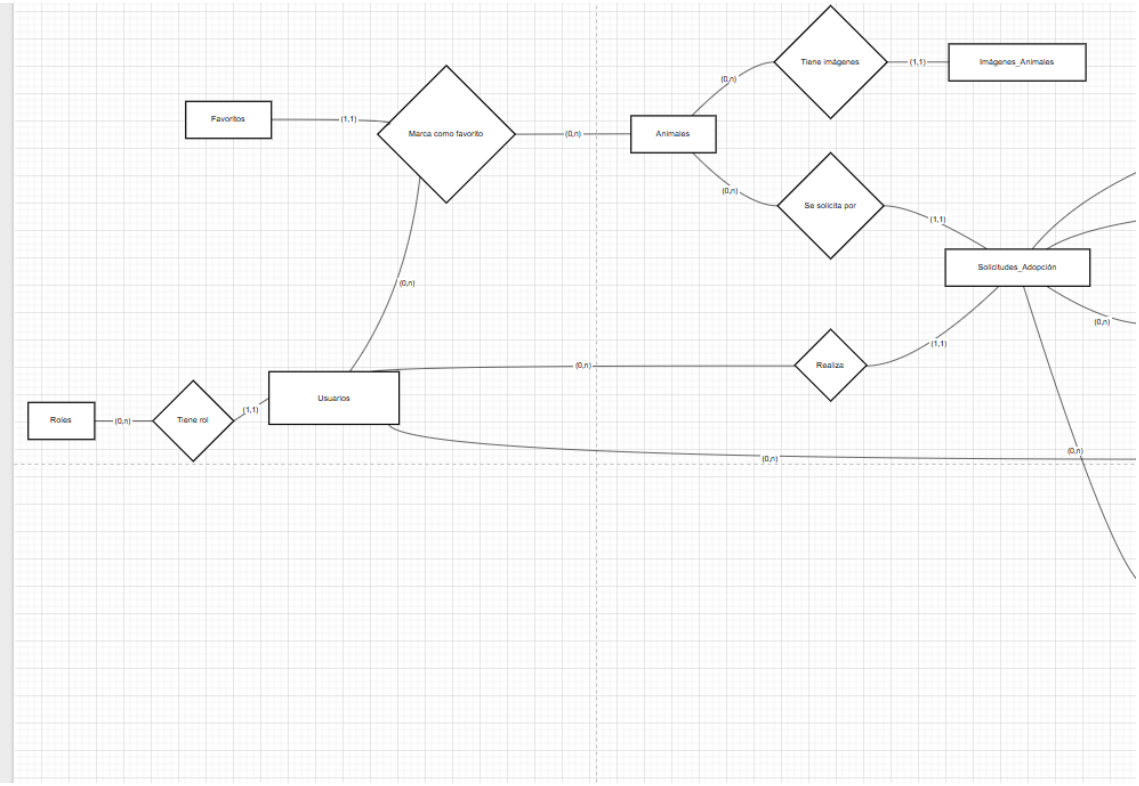
En el diagrama de clases se han utilizado las siguientes notaciones para representar las cardinalidades de las relaciones entre entidades:

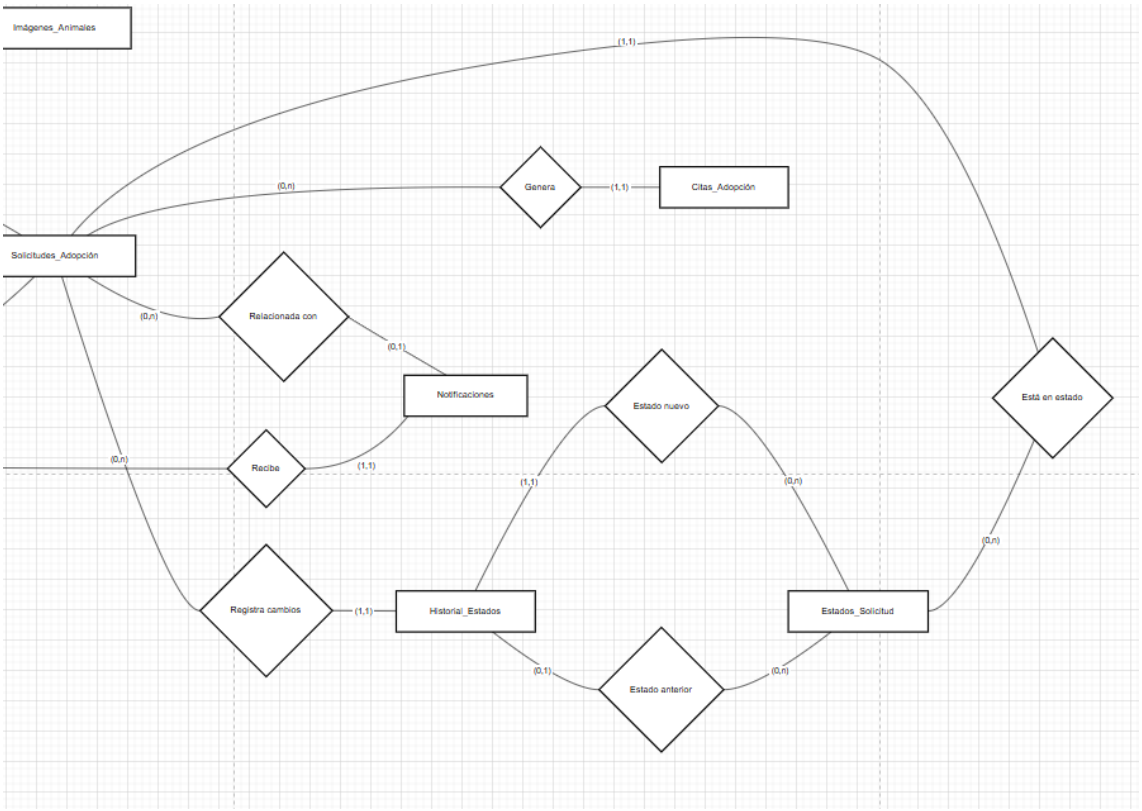
- `1` → exactamente uno (obligatorio)
- `0..1` → cero o uno (opcional)
- `0..*` → de cero a muchos (puede no haber ninguno)
- `1..*` → de uno a muchos (al menos uno)

Estas cardinalidades reflejan las restricciones reales del modelo de datos definidas en el script SQL mediante claves foráneas, claves primarias compuestas y restricciones de unicidad.

5.3. Modelo entidad relación y modelo de base de datos




Favoritos
(1,1)
Marca como favorito
(0,n)
Animales
(0,n)
Tiene imágenes
(1,1)
Imágenes_Animales
(0,n)
Se solicita por
(1,1)
Solicitudes_Adopción
(0,n)
(0,n)
Realiza
(1,1)
Usuarios
(0,n)
Roles
(0,n)
Tiene rol
(1,1)
(0,n)
(0,n)


Imágenes_Animales
(1,1)
(0,n)
Genera
(1,1)
Citas_Adopción
Solicitudes_Adopción
(0,n)
Relacionada con
(0,1)
Notificaciones
Estado nuevo
Está en estado
(0,n)
Recibe
(1,1)
(1,1)
(0,n)
(0,n)
Registra cambios
(1,1)
Historial_Estados
Estados_Solicitud
(0,1)
Estado anterior
(0,n)

6. Arquitectura de la aplicación

La aplicación AdoptaUnCompañero se ha diseñado siguiendo una arquitectura cliente-servidor desacoplada, separando claramente la capa de presentación (frontend) de la capa de lógica de negocio y persistencia (backend). Esta separación permite que ambas partes del sistema evolucionen de forma independiente, facilita el mantenimiento y posibilita que en el futuro se puedan añadir nuevos clientes sin necesidad de modificar el servidor.

La comunicación entre el frontend y el backend se realiza mediante una API REST que devuelve los datos en formato JSON, lo que permite que cualquier cliente capaz de realizar peticiones HTTP pueda consumir los servicios expuestos. Para la autenticación de los usuarios se ha optado por un sistema basado en JWT (JSON Web Tokens), que se ajusta perfectamente al carácter stateless propio de las API REST.

6.1.1. Arquitectura general del sistema

El sistema se compone de tres bloques principales que se comunican entre sí: el cliente (frontend desarrollado con Vue.js), el servidor (backend desarrollado con Spring Boot) y la base de datos (MySQL). El cliente envía peticiones HTTP al servidor, el servidor procesa la lógica de negocio y consulta la base de datos a través de Spring Data JPA, devolviendo la respuesta al cliente en formato JSON.

FRONTEND	BACKEND	BASE DE DATOS
Vue.js 3 + Vite Navegador del usuario Puerto 5173	Spring Boot 3 API REST Puerto 8080	MySQL 8 Motor relacional Puerto 3306
Peticiones HTTP (Axios) → JSON	Spring Data JPA (Hibernate) → SQL	Tablas relacionales Datos persistentes

6.1.2. Arquitectura del backend

El backend se ha desarrollado siguiendo el patrón de arquitectura por capas, una variante del patrón MVC (Modelo-Vista-Controlador) adaptado al desarrollo de API REST. En este caso no existe una capa de vista propiamente dicha, ya que la presentación se delega completamente al frontend; el backend se limita a exponer los datos en formato JSON.

La aplicación se ha estructurado en capas bien diferenciadas, donde cada capa tiene una responsabilidad concreta y solo se comunica con la capa inmediatamente inferior. Esta separación de responsabilidades facilita el mantenimiento, las pruebas unitarias y la reutilización de código.

Capa de presentación (Controladores): Recibe las peticiones HTTP procedentes del frontend, valida los parámetros de entrada y delega el procesamiento a la capa de servicio. Devuelve respuestas en formato JSON con el código HTTP adecuado.

Capa de servicio (Lógica de negocio): Contiene todas las reglas de negocio de la aplicación. Coordina las operaciones entre los distintos repositorios y se encarga de las transformaciones entre entidades y DTOs.

Capa de acceso a datos (Repositorios): Se comunica directamente con la base de datos a través de Spring Data JPA. Proporciona métodos para realizar las operaciones CRUD sobre las entidades del modelo.

Capa de persistencia (Entidades): Define las clases Java que se mapean directamente con las tablas de la base de datos mediante anotaciones JPA.

Capa de seguridad: Gestiona la autenticación y autorización de los usuarios mediante Spring Security y JWT, controlando el acceso a los distintos endpoints según el rol del usuario.

6.1.3. Arquitectura del frontend

El frontend se ha desarrollado utilizando Vue.js 3 con la Composition API, un framework progresivo de JavaScript que permite construir interfaces de usuario mediante componentes reutilizables. La aplicación es una SPA (Single Page Application), lo que significa que toda la navegación se gestiona en el navegador del cliente sin necesidad de recargar la página completa, mejorando notablemente la experiencia de usuario.

La navegación entre las distintas vistas se gestiona mediante Vue Router, mientras que las peticiones HTTP al backend se realizan a través de Axios, encapsuladas en una capa de servicios que centraliza la comunicación con la API REST.

6.1.4. Comunicación entre frontend y backend

La comunicación entre ambas partes del sistema se realiza mediante peticiones HTTP siguiendo el modelo REST. El frontend, ejecutándose en el navegador del cliente, realiza peticiones a los endpoints expuestos por el backend en su puerto 8080. Para evitar problemas de CORS (Cross-Origin Resource Sharing), el backend se ha configurado para aceptar peticiones procedentes del puerto donde se ejecuta el frontend.

El flujo de una operación típica, como por ejemplo el registro de una solicitud de adopción, sería el siguiente: el usuario rellena el formulario en una vista de Vue, al pulsar el botón de envío se invoca al servicio correspondiente que utiliza Axios para enviar una petición POST al backend. El interceptor de Axios añade automáticamente el token JWT en la cabecera Authorization. El filtro JwtAuthenticationFilter del backend valida el token, y si es correcto, la petición llega al controlador, que delega en el servicio, este consulta el repositorio y finalmente la respuesta vuelve al frontend en formato JSON.

6.2. Estructura del proyecto

El proyecto AdoptaUnCompañero está organizado en tres carpetas principales en la raíz del repositorio, una para cada uno de los componentes del sistema:

BBDD: contiene el script SQL completo para crear la base de datos (BaseDeDatosAdoptaUnCompañero.sql) con todas las tablas, índices, restricciones y datos iniciales necesarios para el funcionamiento de la aplicación. También incluye un Dockerfile para levantar un contenedor de MySQL.

backend/AdoptaUnCompaneroApplication: proyecto Spring Boot que implementa toda la lógica del servidor y la API REST. Sigue la estructura estándar de un proyecto Maven.

frontend/adopta-frontend: proyecto Vue.js que implementa la interfaz de usuario. Sigue la estructura estándar de un proyecto creado con Vite.

6.2.1. Estructura del backend

El código fuente del backend se ha organizado en los siguientes paquetes dentro de com.daw.adoptauncompanero, siguiendo una arquitectura por capas:

Paquete	Descripción
config	Clases de configuración global de la aplicación: configuración de CORS para permitir peticiones desde el frontend, configuración para servir archivos estáticos (imágenes subidas), configuración del envío de emails y otros beans de Spring.
controladores	Clases @RestController que exponen los endpoints de la API REST. Reciben las peticiones HTTP, validan los datos de entrada y delegan en los servicios. Cada controlador agrupa las operaciones de una entidad concreta.
dtos	Data Transfer Objects. Clases que se utilizan para transportar los datos entre el frontend y el backend, evitando exponer directamente las entidades de la base de datos.
entities	Entidades JPA (@Entity) que representan las tablas de la base de datos: Usuario, Rol, Animal, ImagenAnimal, SolicitudAdopcion, EstadoSolicitud, HistorialEstadoSolicitud, Favorito, CitaAdopcion, Notificacion y Noticia.
repositorios	Interfaces que extienden de JpaRepository de Spring Data JPA. Definen los métodos de acceso a datos para cada entidad, incluyendo consultas personalizadas mediante @Query cuando es necesario.
security	Configuración de Spring Security: clase SecurityConfig, JwtTokenProvider para generar y validar tokens JWT, JwtAuthenticationFilter que intercepta las peticiones para validar el token, y CustomUserDetailsService que carga los datos del usuario desde la base de datos.
servicio	Clases @Service que contienen la lógica de negocio. Coordinan las operaciones entre los distintos repositorios, aplican las reglas de negocio y gestionan las transacciones.

6.2.2. Estructura del frontend

La estructura del proyecto frontend sigue las convenciones de un proyecto Vue creado con Vite, con las siguientes carpetas dentro de src:

Carpeta	Descripción

assets	Recursos estáticos como imágenes, iconos y hojas de estilo CSS globales utilizadas en toda la aplicación.
components	Componentes Vue reutilizables que se usan en distintas vistas: HeaderMenu, PieContacto, TarjetaAnimal, TablaSolicitudes y BarraPublicidad.
router	Configuración de Vue Router con las rutas de la aplicación y los guards de navegación que protegen las rutas que requieren autenticación o un rol específico.
services	Capa que encapsula todas las llamadas HTTP al backend. Contiene un servicio por cada entidad: animalService, authService, citaService, favoritoService, noticiaService, solicitudService y usuarioService. El archivo api.js configura la instancia de Axios con la URL base y los interceptores que añaden el JWT a cada petición.
stores	Almacén del estado global de la aplicación: usuario autenticado, token JWT, lista de animales en favoritos, etc.
views	Vistas principales de la aplicación que se cargan según la ruta activa: Home, Login, Registro, ListadoAnimales, DetalleAnimal, MisSolicitudes, PanelAdmin, etc.

6.3. Librerías externas utilizadas

A continuación se detallan las principales librerías y dependencias externas utilizadas en el proyecto, tanto en el backend como en el frontend.

6.3.1. Librerías del backend (definidas en pom.xml)

Dependencia	Función
spring-boot-starter-web	Starter de Spring Boot que incluye Spring MVC, Tomcat embebido y Jackson para el procesamiento de JSON. Permite crear aplicaciones web y API REST.
spring-boot-starter-data-jpa	Starter para el acceso a datos mediante JPA. Incluye Spring Data JPA, Hibernate como implementación y un pool de conexiones HikariCP.
spring-boot-starter-security	Starter de Spring Security para gestionar la autenticación y autorización. Incluye BCrypt para el hash de contraseñas.
spring-boot-starter-validation	Permite validar los DTOs mediante anotaciones (@NotNull, @Email, @Size, etc.) usando Hibernate Validator.
spring-boot-starter-mail	Proporciona JavaMailSender para el envío de correos electrónicos a través de SMTP.
mysql-connector-j	Driver JDBC oficial de MySQL para conectar la aplicación con la base de datos.

jjwt-api / jjwt-impl / jjwt-jackson	Librería JJWT (Java JWT), utilizada para generar y validar los tokens JWT.
spring-boot-devtools	Herramientas de desarrollo que aceleran el ciclo de desarrollo: reinicio automático al detectar cambios y recarga en caliente.
spring-boot-starter-test	Starter para pruebas que incluye JUnit 5, Mockito, AssertJ y Spring Test.

6.3.2.Librerías del frontend (definidas en package.json)

Dependencia	Función
vue	El propio framework Vue.js 3 con la Composition API.
vue-router	Router oficial de Vue para gestionar la navegación entre las distintas vistas de la SPA.
axios	Cliente HTTP basado en promesas para realizar las peticiones a la API REST del backend.
vite	Build tool y servidor de desarrollo con Hot Module Replacement.
@vitejs/plugin-vue	Plugin de Vite para procesar los archivos .vue (Single File Components).
eslint / eslint-plugin-vue	Linter para detectar errores y problemas de estilo en el código JavaScript y Vue.
prettier	Formateador automático de código que asegura un estilo consistente en todo el proyecto.

7. Manual de despliegue

Demostrar el despliegue con Docker

1. Mostrar las imágenes Docker creadas

Comando:

docker images

```
PS C:\Users\Diego\Desktop\TFG> docker images
                                                    i Info →   U  In Use
IMAGE                                ID            DISK USAGE   CONTENT SIZE   EXTRA
diegolarra06/adopta-backend:1.0      e22d0b013c02       527MB         157MB     U
diegolarra06/adopta-db:1.0           08be41f423a3       1.1GB         238MB     U
diegolarra06/adopta-frontend:1.0
```

Muestra las imágenes Docker que se han construido para la aplicación. En mi caso he creado tres imágenes: una para la base de datos MySQL, otra para el backend Spring Boot y otra para el frontend Vue con Nginx.

2. Mostrar la red Docker creada

Comando:

docker network ls

```
PS C:\Users\Diego\Desktop\TFG> docker network ls
NETWORK ID     NAME         DRIVER    SCOPE
e1ea9421b516   adopta-red   bridge    local
2907cf47a985   bridge       bridge    local
```

He creado una red propia llamada adopta-red para que los contenedores puedan comunicarse entre ellos por nombre. Por ejemplo, el backend se conecta a la base de datos usando el nombre del contenedor mysql-adopta.

3. Mostrar los volúmenes creados

Comando:

docker volume ls

```
PS C:\Users\Diego\Desktop\TFG> docker volume ls
DRIVER    VOLUME NAME
local     adopta_mysql_data
local     adopta_uploads
local     volumen1
```

He creado dos volúmenes: uno para guardar los datos de MySQL y otro para guardar los archivos subidos por la aplicación. Así, aunque se borren los contenedores, los datos importantes pueden mantenerse.

4. Arrancar la base de datos

Si el contenedor ya existe, puedes arrancarlo con:

docker run -d --name mysql-adopta --network adopta-red -p 3307:3306 -v adopta_mysql_data:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=PracticaRoot diegolarra06/adopta-db:entrega-final

Este comando arranca el contenedor de MySQL.

La opción --name mysql-adopta le da un nombre al contenedor.

La opción --network adopta-red lo conecta a la red común.

La opción -p 3307:3306 publica MySQL en el puerto 3307 del ordenador porque el puerto 3306 ya estaba ocupado.

La opción -v adopta_mysql_data:/var/lib/mysql guarda los datos de MySQL en un volumen.

La variable MYSQL_ROOT_PASSWORD=PracticaRoot establece la contraseña de root de MySQL.

5. Comprobar logs de MySQL

Comando:

docker logs mysql-adopta

```
2026-05-09T15:30:39.270877Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
2026-05-09T15:30:39.540623Z 0 [Warning] [MY-010068] [Server] CA certificate ca.pem is self signed.
2026-05-09T15:30:39.540671Z 0 [System] [MY-013602] [Server] Channel mysql_main configured to support TLS. Encrypted connections are now supported
2026-05-09T15:30:39.543170Z 0 [Warning] [MY-011810] [Server] Insecure configuration for --pid-file: Location '/var/run/mysqld' in the path is acc
2026-05-09T15:30:39.562868Z 0 [System] [MY-011323] [Server] X Plugin ready for connections. Socket: /var/run/mysqld/mysqlx.sock
2026-05-09T15:30:39.563027Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. Version: '8.4.9'  socket: '/var/run/mysqld/m
2026-05-09 15:30:39+00:00 [Note] [Entrypoint]: Temporary server started.
'/var/lib/mysql/mysql.sock' -> '/var/run/mysqld/mysqld.sock'
Warning: Unable to load '/usr/share/zoneinfo/iso3166.tab' as time zone. Skipping it.
Warning: Unable to load '/usr/share/zoneinfo/leap-seconds.list' as time zone. Skipping it.
Warning: Unable to load '/usr/share/zoneinfo/leapseconds' as time zone. Skipping it.
Warning: Unable to load '/usr/share/zoneinfo/tzdata.zi' as time zone. Skipping it.
Warning: Unable to load '/usr/share/zoneinfo/zone.tab' as time zone. Skipping it.
Warning: Unable to load '/usr/share/zoneinfo/zone1970.tab' as time zone. Skipping it.
2026-05-09 15:30:41+00:00 [Note] [Entrypoint]: Creating database adopta_un_companero

2026-05-09 15:30:41+00:00 [Note] [Entrypoint]: /usr/local/bin/docker-entrypoint.sh: running /docker-entrypoint-initdb.d/01_init.sql


2026-05-09 15:30:41+00:00 [Note] [Entrypoint]: Stopping temporary server
2026-05-09T15:30:41.676840Z 12 [System] [MY-013172] [Server] Received SHUTDOWN from user root. Shutting down mysqld (Version: 8.4.9).
2026-05-09T15:30:42.908785Z 0 [System] [MY-010910] [Server] /usr/sbin/mysqld: Shutdown complete (mysqld 8.4.9)  MySQL Community Server - GPL.
2026-05-09T15:30:42.908820Z 0 [System] [MY-015016] [Server] MySQL Server - end.
2026-05-09 15:30:43+00:00 [Note] [Entrypoint]: Temporary server stopped

2026-05-09 15:30:43+00:00 [Note] [Entrypoint]: MySQL init process done. Ready for start up.

2026-05-09T15:30:43.695712Z 0 [System] [MY-015015] [Server] MySQL Server - start.
2026-05-09T15:30:43.900297Z 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 8.4.9) starting as process 1
2026-05-09T15:30:43.904726Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
2026-05-09T15:30:44.111153Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
2026-05-09T15:30:44.331024Z 0 [Warning] [MY-010068] [Server] CA certificate ca.pem is self signed.
2026-05-09T15:30:44.331063Z 0 [System] [MY-013602] [Server] Channel mysql_main configured to support TLS. Encrypted connections are now supported
2026-05-09T15:30:44.333517Z 0 [Warning] [MY-011810] [Server] Insecure configuration for --pid-file: Location '/var/run/mysqld' in the path is acc
2026-05-09T15:30:44.360923Z 0 [System] [MY-011323] [Server] X Plugin ready for connections. Bind-address: '::' port: 33060, socket: /var/run/mysq
2026-05-09T15:30:44.361149Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. Version: '8.4.9'  socket: '/var/run/mysqld/m
PS C:\Users\Diego\Desktop\TFG>
```

Se comprueba que la base de datos ha arrancado correctamente y que se ha importado el script SQL inicial.

6. Arrancar el backend

Si el contenedor ya existe:

docker start backend-adopta

Comando completo usado para crearlo:

docker run -d --name backend-adopta --network adopta-red -p 8080:8080 -v adopta_uploads:/app/uploads -e SPRING_DATASOURCE_URL="jdbc:mysql://mysql-adopta:3306/adopta_un_companero?useSSL=false&serverTimezone=UTC&characterEncoding=UTF-8&allowPublicKeyRetrieval=true" -e SPRING_DATASOURCE_USERNAME=root -e SPRING_DATASOURCE_PASSWORD=PracticaRoot diegolarra06/adopta-backend:entrega-final

Este comando arranca el backend desarrollado con Spring Boot.

La opción --network adopta-red permite que el backend pueda conectarse a MySQL.

La conexión a la base de datos se pasa con la variable SPRING_DATASOURCE_URL.

En vez de usar localhost, se usa mysql-adopta, que es el nombre del contenedor de la base de datos dentro de la red Docker.

El puerto 8080:8080 permite acceder al backend desde el ordenador en:

http://localhost:8080

7. Comprobar logs del backend

Comando:

docker logs backend-adopta

```
PS C:\Users\Diego\Desktop\TFG> docker logs backend-adopta

  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/

 :: Spring Boot ::                (v3.3.5)

2026-05-09T15:31:01.766Z  INFO 1 --- [AdoptaUnCompaneroApplication] [           main] c.d.a.AdoptaUnCompaneroApplication      : Starting AdoptaUnCompaneroApplication v0.0.1-SNAPSHOT using Java 17.0.19 with PID 1 (/app/
app.jar started by root in /app)
2026-05-09T15:31:01.770Z  INFO 1 --- [AdoptaUnCompaneroApplication] [           main] c.d.a.AdoptaUnCompaneroApplication      : No active profile set, falling back to 1 default profile: "default"
2026-05-09T15:31:02.937Z  INFO 1 --- [AdoptaUnCompaneroApplication] [           main] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2026-05-09T15:31:03.091Z  INFO 1 --- [AdoptaUnCompaneroApplication] [           main] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 139 ms. Found 11 JPA repository interfaces.
2026-05-09T15:31:04.286Z  INFO 1 --- [AdoptaUnCompaneroApplication] [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat initialized with port 8080 (http)
2026-05-09T15:31:04.308Z  INFO 1 --- [AdoptaUnCompaneroApplication] [           main] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
2026-05-09T15:31:04.309Z  INFO 1 --- [AdoptaUnCompaneroApplication] [           main] o.apache.catalina.core.StandardEngine    : Starting Servlet engine: [Apache Tomcat/10.1.31]
2026-05-09T15:31:04.390Z  INFO 1 --- [AdoptaUnCompaneroApplication] [           main] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring embedded WebApplicationContext
2026-05-09T15:31:04.393Z  INFO 1 --- [AdoptaUnCompaneroApplication] [           main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 2553 ms
2026-05-09T15:31:04.698Z  INFO 1 --- [AdoptaUnCompaneroApplication] [           main] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Starting...
2026-05-09T15:31:04.951Z  INFO 1 --- [AdoptaUnCompaneroApplication] [           main] com.zaxxer.hikari.pool.HikariPool        : HikariPool-1 - Added connection com.mysql.cj.jdbc.ConnectionImpl@03501be
2026-05-09T15:31:04.954Z  INFO 1 --- [AdoptaUnCompaneroApplication] [           main] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Start completed.
2026-05-09T15:31:05.003Z  INFO 1 --- [AdoptaUnCompaneroApplication] [           main] o.hibernate.jpa.internal.util.LogHelper  : HHH000204: Processing PersistenceUnitInfo [name: default]
2026-05-09T15:31:05.100Z  INFO 1 --- [AdoptaUnCompaneroApplication] [           main] org.hibernate.Version                    : HHH000412: Hibernate ORM core version 6.5.3.Final
2026-05-09T15:31:05.142Z  INFO 1 --- [AdoptaUnCompaneroApplication] [           main] o.h.c.internal.RegionFactoryInitiator    : HHH000026: Second-level cache disabled
```

permite comprobar que el backend ha arrancado y que ha conectado correctamente con la base de datos.

8. Arrancar el frontend

Si el contenedor ya existe:

docker start frontend-adopta

Comando completo usado para crearlo:

docker run -d --name frontend-adopta --network adopta-red -p 80:80 diegolarra06/adopta-frontend:entrega-final

Explicación: Este comando arranca el frontend de la aplicación. El frontend está construido con Vue y se sirve mediante Nginx.

9. Comprobar que los tres contenedores están activos

Comando:

docker ps

Explicación: muestra los contenedores que están actualmente en ejecución. Deben aparecer los tres contenedores principales de la aplicación.

```
PS C:\Users\Diego\Desktop\TFG> docker ps
CONTAINER ID   IMAGE                        COMMAND              CREATED        STATUS        PORTS                                      NAMES
17dbaa0a0ff4   diegolarra06/adopta-backend:1.0   "java -jar app.jar"   18 minutes ago   Up 18 minutes   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp   backend-adopta
b55b14f34810   diegolarra06/adopta-db:1.0        "docker-entrypoint.s…"   18 minutes ago   Up 18 minutes   0.0.0.0:3307->3306/tcp, [::]:3307->3306/tcp   mysql-adopta
ab2fdbc42d86   diegolarra06/adopta-frontend:1.0  "/docker-entrypoint.…"   4 hours ago    Up 4 hours    0.0.0.0:80->80/tcp, [::]:80->80/tcp        frontend-adopta
PS C:\Users\Diego\Desktop\TFG>
```

10. Abrir la aplicación en el navegador

11. Comprobar que el frontend se comunica con el backend

1. Busca peticiones a:

http://localhost:8080/api/animales

Aquí se comprueba que el frontend está llamando correctamente al backend. Las peticiones deben devolver estado 200, lo que significa que la comunicación entre frontend, backend y base de datos funciona correctamente.

12. Comandos para parar la aplicación

Comando:

docker stop frontend-adopta backend-adopta mysql-adopta

detiene los tres contenedores de la aplicación. Se detiene primero el frontend, luego el backend y finalmente la base de datos.

13. Comandos para volver a abrir la aplicación

Comandos:

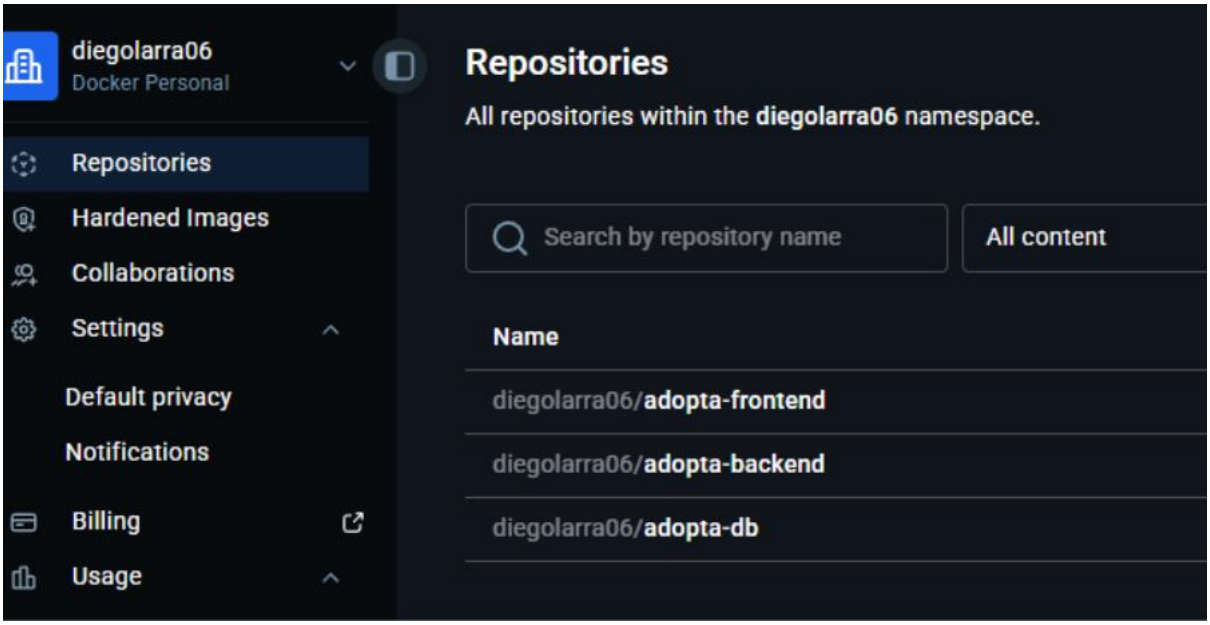
docker start mysql-adopta

docker start backend-adopta

docker start frontend-adopta

```
PS C:\Users\Diego\Desktop\TFG> docker start mysql-adopta
mysql-adopta
PS C:\Users\Diego\Desktop\TFG> docker start backend-adopta
backend-adopta
PS C:\Users\Diego\Desktop\TFG> docker start frontend-adopta
frontend-adopta
PS C:\Users\Diego\Desktop\TFG> docker ps
```

Si los contenedores ya existen, no es necesario volver a crearlos. Basta con arrancarlos de nuevo en orden: primero la base de datos, después el backend y finalmente el frontend.

14. Subir a DockerHub

```
PS C:\Users\Diego\Desktop\TFG> docker push diegolarra06/adopta-db:entrega-final
The push refers to repository [docker.io/diegolarra06/adopta-db]
56b4ed70176d: Pushed
1ba345b2c7a5: Pushed
c14e3f115d16: Pushed
3221f853f73e: Pushed
68c4d53bd503: Pushed
fc828cf283e3: Pushed
6405a7c8d60b: Pushed
35ff074259fa: Pushed
edf85873f64e: Pushed
0d744359c8b9: Pushed
344c14a42aea: Pushed
a85f525ae670: Pushed
entrega-final: digest: sha256:08be41f423a30995e20112c165f3c4019f4627e0e336a7bf2a640d5fe1e31a33 size: 856
PS C:\Users\Diego\Desktop\TFG> docker push diegolarra06/adopta-backend:entrega-final
The push refers to repository [docker.io/diegolarra06/adopta-backend]
c7bfdab0a425: Pushed
```